Gear-Shifting Control for Reduced Steady-State Jitter in an All-Digital Phase-Locked Loop

Magesh B, Allen Joe A, Venkata Sai Neeraj
Department of Electronics and Communication Engineering, VIT University, Chennai

Abstract*—In an all-digital phase-locked loop (ADPLL), a wide loop bandwidth locks quickly but leaves the digitally controlled oscillator (DCO) hunting between adjacent quantization levels, which appears as cycle-to-cycle (C2C) jitter. A narrow bandwidth lowers that jitter but slows acquisition. This paper presents a gear-shifting ADPLL (GS-ADPLL) in which a single synthesizable lock-state finite state machine (FSM) changes the loop's behavior once lock is confirmed. After frequency lock, the FSM raises the loop-filter threshold from Gear 1 to Gear 2 and lets the filter compare phase only once in every K reference cycles. We swept the feedback division ratio (M = 1, 2, 4), the duty factor (K = 1 to 16) and the Gear 2 threshold (4 to 14) in RTL simulation. At K = 8, steady-state C2C jitter falls from 38.4 ps to 3.1 ps for M = 1, from 18.8 ps to 1.6 ps for M = 2, and from 7.8 ps to 0.8 ps for M = 4 (reductions of 91.93%, 91.49% and 89.74%), while loop-filter activity drops by 87.5% and lock time is unchanged. Synthesis in Cadence Genus shows a 63.66% area increase (537.399 to 879.518 µm²) and a shorter critical path (725 ps versus 1064 ps). Vector-based analysis in Cadence Voltus puts the total power increase at only 1.48% (0.585274 to 0.593940 mW), because gating the loop filter offsets the power of the added control logic.

***Index Terms*—All-digital phase-locked loop (ADPLL), gear shifting, cycle-to-cycle jitter, digitally controlled oscillator (DCO), digital loop filter, finite state machine (FSM), ASIC synthesis, power analysis, physical design.**

I. Introduction

Phase-locked loops (PLLs) generate and clean up the clocks that nearly every system-on-chip (SoC) relies on [1], [2]. Analog charge-pump PLLs have long been the first choice because of their low phase noise. In deep sub-micron CMOS they become harder to build: supply headroom shrinks, intrinsic transistor gain drops, gate leakage rises, and the circuits grow sensitive to process, voltage and temperature (PVT) variation [3], [4].

All-digital PLLs (ADPLLs) avoid most of these problems [4]–[6]. The phase detector, loop filter and control path are built from logic gates and flip-flops, and the voltage-controlled oscillator is replaced by a digitally controlled oscillator (DCO) whose period is set by a digital tuning word [2], [7]. Such a design goes through a standard-cell ASIC flow, moves easily between process nodes, stores its state digitally and can be reconfigured at run time [3], [8].

A fixed-parameter ADPLL still has to trade acquisition speed against steady-state jitter [8], [9]. Lock needs a wide loop bandwidth so that the initial frequency error disappears within a few reference cycles. Once the loop is locked, that same bandwidth becomes a liability. DCO tuning is quantized, so every update moves the oscillator period by a finite step, and the target frequency usually falls between two DCO levels. The loop therefore keeps toggling the control word between neighboring codes, a behavior known as limit-cycle hunting, and the output clock picks up cycle-to-cycle (C2C) jitter [2], [9].

This paper addresses the trade-off with a gear-shifting ADPLL (GS-ADPLL). A lock-state finite state machine (FSM) runs the loop in an aggressive acquisition mode (Gear 1) until lock has been confirmed, then moves it into a quieter, duty-cycled tracking mode (Gear 2). The main contributions are:

- A single synthesizable lock-state FSM that drives two coordinated controls: loop-filter bandwidth switching and 1-in-K duty-cycled comparison gating.
- A parametric jitter study over division ratios (M = 1, 2, 4), duty factors (K = 1 to 16) and Gear 2 thresholds (`CNT_N_GEAR2` = 4 to 14). Jitter falls by up to 96.88%, and K = 8 emerges as the best operating point (89.74% to 91.93% reduction with 87.5% less filter activity).
- An ASIC implementation in Cadence Genus and Innovus, together with an explanation of why the synthesis-level power estimate (+64.50%) differs so much from the vector-based Voltus result (+1.48%).

The rest of the paper is organized as follows. Section II gives background and related work, Section III describes the baseline ADPLL, and Section IV the proposed GS-ADPLL. Section V covers RTL simulation, Section VI synthesis and timing, Section VII power, and Section VIII physical design. Section IX presents the jitter results and parameter sweeps, Section X discusses trade-offs and limitations, and Section XI concludes.

II. Background and Related Work

A. ADPLL Operating Principles and Loop Dynamics

A classical ADPLL is a negative-feedback loop built from a phase/frequency detector (PFD), a digital loop filter, a DCO and a programmable frequency divider (1/M) [1], [4]. The PFD compares the reference clock `REF_CLK` (frequency f_{REF}) with the divided feedback clock `FB` (frequency f_{FB}) and issues up and down pulses (`p_up` and `p_down`) that give the sign of the error.

The loop transfer function, in the s or z domain, sets the closed-loop bandwidth ω_n and the damping factor ζ [2], [8]. A high bandwidth corrects a large initial frequency offset quickly,

but it also lets more phase-detector and quantization noise reach the output clock. With static loop parameters, a high ω_n therefore gives fast lock and the largest steady-state jitter [9].

B. DCO Quantization Noise and Limit-Cycle Hunting

An analog LC-VCO is tuned by a continuous control voltage. A DCO instead changes its frequency through a discrete digital word D[n] [7], so its output frequency can be written as

$$f_{OUT}[n] = f_0 + K_{DCO} \cdot D[n] \quad (1)$$

where f_0 is the free-running frequency and K_{DCO} is the frequency step per least-significant bit (LSB). The target frequency $f_{target} = M \cdot f_{REF}$ rarely lands exactly on a DCO level, so the loop cannot settle on one control word. The PFD keeps detecting a small residual phase error, and the controller keeps switching between D[n] and D[n] ± 1 [2], [4]. This repeated crossing of a quantization boundary is limit-cycle hunting.

Each ±1 change in D[n] alters the delay path inside the DCO and shifts the instantaneous period of `OUT_CLK`. C2C jitter, the difference between adjacent clock periods, therefore grows with how often the DCO word is updated. A loop filter that updates the DCO on every reference cycle gives the largest C2C variation [8].

C. Adaptive Bandwidth and Gear-Switching Approaches

Adaptive-bandwidth and gear-shifting techniques have been studied in both analog and digital PLLs [3], [8], [10]. In analog charge-pump loops, Hanumolu et al. [10] used a dual-path loop filter and reduced the charge-pump current after acquisition. Changing currents or filter capacitances in continuous time injects charge, however, which disturbs the phase and can cause cycle slips [10].

In the digital domain, gear shifting comes down to register multiplexing and state-machine control. Staszewski and Balsara [4] applied variable loop-gain scaling in digital RF synthesizers, and Chung and Lee [5] switched between coarse and fine delay lines. Liu and Ai [3] showed that low switching activity and low supply voltage matter most when an IoT design has a sub-milliwatt power budget. Many digital schemes, though, shift gears open-loop or on a timer, or handle bandwidth adjustment separately from activity gating. The GS-ADPLL instead combines a two-level filter threshold and duty-cycled comparison in one lock-state FSM that is driven by the loop's own lock indication.

III. Baseline ADPLL Architecture

A conventional synthesizable ADPLL served as the baseline. It has four modules: a phase/frequency detector (`PFD.v`), a controller (`CONTROLLER.v`) that contains a successive-approximation acquisition engine and a Motorola-style digital loop filter (`FILTER.v`), a DCO (`DCO.v`), and a programmable Johnson-counter feedback divider (`FREQ_DIV.v`).

A. Phase/Frequency Detector (PFD)

The PFD compares `REF_CLK` with the feedback clock `Out_divM`. It is built from flip-flops with an asynchronous reset path. It raises `flagu` (phase up) when `REF_CLK` leads `Out_divM` and `flagd` (phase down) when `Out_divM` leads. The OR of the two flags forms the `phase_clk` strobe, which triggers the digital comparison in the controller.

B. Controller and Successive-Approximation Acquisition

After reset, the controller finds the target frequency with a binary search. It starts at `dco_code_int` = 32 with a step of 8. At each phase comparison, `dco_code_int` moves by ±`step` according to the sign of the phase error, which is tracked in `p_history`. When the direction reverses, the oscillator has passed the target and the step is halved (8, 4, 2, 1). Once the step reaches 1, coarse acquisition is finished and the controller asserts `freq_lock`.

C. Motorola Digital Loop Filter

After `freq_lock` is asserted, the digital loop filter takes over tracking. It follows the dual-counter structure of the Motorola patent (US 5,506,875) [9]. Two 4-bit saturating counters, `cnt_up` and `cnt_down`, count consecutive phase-lead and phase-lag cycles. Only when one of them reaches the threshold `CNT_N` does the filter move the running average code `avg_code` by ±1 LSB. In the baseline, `CNT_N` is fixed at 4 and the comparison runs on every active reference cycle.

D. Digitally Controlled Oscillator (DCO) and Divider

The 8-bit `avg_code` is converted to a 129-bit thermometer code (`code[128:0]`) by a combinational decoder. Thermometer coding keeps the tuning monotonic and avoids the glitches of binary-weighted tuning. The DCO is a ring oscillator built from tri-state inverter delay stages, following Hsu et al. [7]. Enabling more code lines changes the capacitive loading and electrical length of the ring, which sets the output period over 128 discrete steps. The divider (`FREQ_DIV.v`) divides `OUT_CLK` by M (1, 2 or 4) to create `Out_divM`.

IV. Proposed GS-ADPLL Architecture

The GS-ADPLL adds two coordinated mechanisms to the baseline loop. Both are controlled by one single-clock-domain lock-state FSM (`lock_detector_fsm.v`): Mechanism 1, adaptive loop-filter bandwidth (gear) switching, and Mechanism 2, duty-cycled comparison gating. Fig. 1 shows the complete architecture.

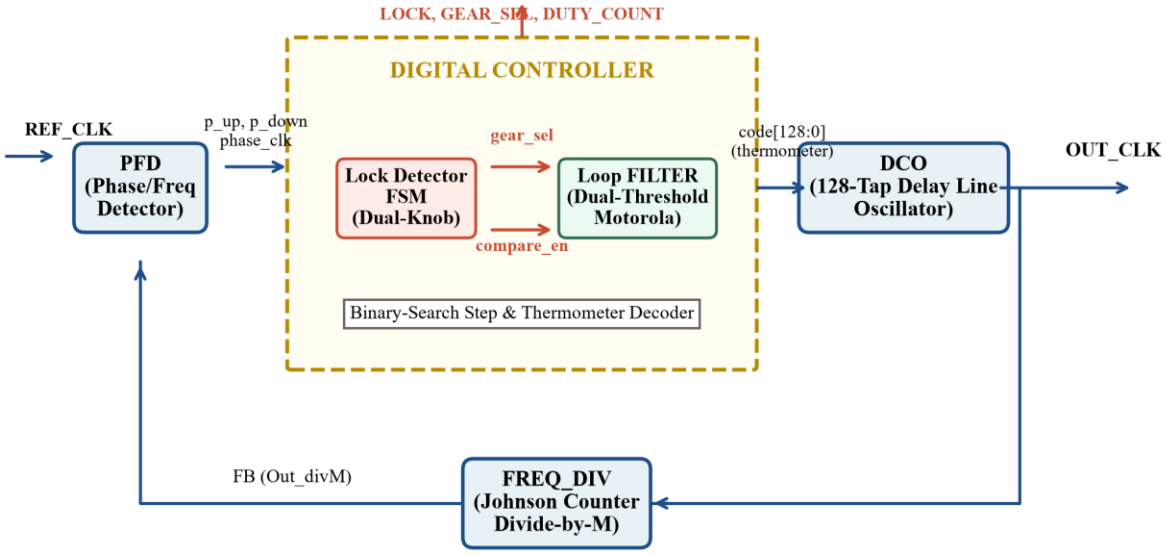


Fig. 1. Top-level block diagram of the proposed GS-ADPLL, showing the lock-detector FSM, dual-threshold loop filter, 129-bit thermometer decoder, structural DCO delay chain and feedback divider.

A. Lock-State FSM Design

The `lock_detector_fsm` module is clocked on the falling edge of `ref_clk` and cleared asynchronously by reset. It has two states, `S_ACQUIRE` (state 0) and `S_TRACK` (state 1), as shown in Fig. 2.

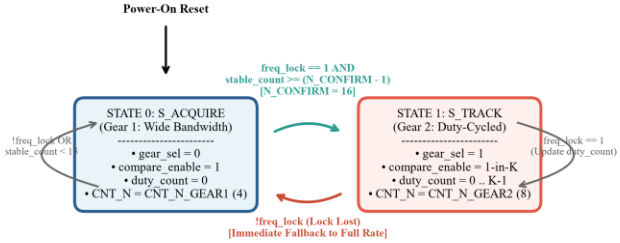


Fig. 2. State transition diagram of the lock-detector FSM: acquisition (Gear 1), the `N_CONFIRM` stability count, duty-cycled tracking (Gear 2) and immediate fallback on loss of lock.

1) State S_ACQUIRE (Gear 1). After reset release the FSM enters `S_ACQUIRE`. Here `gear_sel` is held at 0 and `compare_enable` stays at 1, so acquisition runs at the full reference rate with the wide-bandwidth setting (`CNT_N_GEAR1` = 4), exactly as in the baseline. An 8-bit counter, `stable_count`, watches `freq_lock` so that a momentary glitch cannot trigger an early gear change. The FSM moves to `S_TRACK` only after `freq_lock` has stayed asserted for `N_CONFIRM` = 16 consecutive reference cycles:

$$T_{confirm} = N_{CONFIRM} \cdot T_{REF} = 16 \cdot T_{REF} \qquad (2)$$

2) State S_TRACK (Gear 2). Once stability is confirmed, the FSM enters `S_TRACK`. The signal `gear_sel` goes to 1 and the loop filter raises its threshold to `CNT_N_GEAR2` = 8. At the same time a 4-bit modulo-K counter, `duty_count`, starts. `compare_enable` is asserted for one reference cycle when `duty_count` reaches `DUTY_K` − 1 and is low for the other K − 1 cycles. While it is low, the loop filter freezes its counters and holds `avg_code`, so the DCO word does not toggle.

3) Re-acquisition fallback. If a large reference step, a phase disturbance or a supply glitch makes `freq_lock` drop while in `S_TRACK`, the FSM returns to `S_ACQUIRE` within one clock cycle. The signal `gear_sel` goes back to 0 and `compare_enable` is forced to 1, so re-acquisition is never slowed down by duty-cycled sampling.

B. Mechanism 1: Loop-Filter Bandwidth Gear Shifting

Inside `FILTER.v`, the active threshold is selected by `gear_sel`:

$$N_{th} = N_{G2} \text{ for } g = 1; \; N_{G1} \text{ for } g = 0 \qquad (3)$$

where N_{th} is `cnt_n_thresh`, g is `gear_sel`, N_{G1} = `CNT_N_GEAR1` and N_{G2} = `CNT_N_GEAR2`. Going from 4 to 8 in Gear 2 means that the filter needs twice as many consecutive errors in the same direction before it moves the DCO code. This acts as a strong low-pass filter on phase-detector quantization noise and directly weakens limit-cycle hunting.

C. Mechanism 2: Duty-Cycled Comparison and Activity Suppression

In Gear 2, `compare_enable` gates every register update in the loop filter. The ideal reduction in tracking activity is

$$\textit{Activity reduction (\%)} = 100 \times (1 - 1/K) \quad (4)$$

At the chosen point K = 8, the comparison runs once in every 8 reference cycles (duty ratio 0.125), which gives an ideal activity reduction of 87.5%. Updating the DCO this much less often is the physical reason for the jitter reduction reported in Section IX.

V. RTL Design and Functional Verification

The baseline and modified designs were compared side by side with Cadence NC-Verilog/Xcelium and Icarus Verilog. Both testbenches (`baseline_TEST.v` and `modified_TEST.v`) include the same measurement library (`common/jitter_tasks.vh`), so stimulus, measurement window and calculations are identical.

A. Measurement Task Implementation

After lock and a settling margin, the measurement task captures `JITTER_SAMPLES` = 256 consecutive `OUT_CLK` cycles (N = 256) and computes the mean C2C jitter as

$$J_{CC,mean} = 1/(N-1) \cdot \Sigma\, |T_{OUT}[n] - T_{OUT}[n-1]| \quad (5)$$

Long-term period drift is the difference between the mean period of the second half (2nd) of the capture window and that of the first half (1st):

$$\Delta T_{drift} = (T_{mean,2nd} - T_{mean,1st}) \times 1000\ ps \quad (6)$$

B. Waveform Interpretation and Gear Transition

GTKWave captures confirm the gear-shifting sequence. In the baseline waveform, `phase_clk` pulses continuously during both acquisition and steady state. Because `CNT_N` is fixed at 4, the DCO code toggles often between adjacent values, which gives high C2C dispersion.

The GS-ADPLL waveform shows the following sequence. (1) After reset release, coarse acquisition halves `step` from 8 to 1. (2) When `step` = 1, `freq_lock` asserts. (3) `stable_count` counts from 0 to 15 while `gear_sel` stays at 0 and `compare_enable` stays at 1. (4) At the 16th stable cycle the gear shift occurs: `gear_sel` goes high and `duty_count` starts cycling from 0 to 7. From then on, `compare_enable` pulses once every 8 cycles and `avg_code` stays flat in between. The DCO control word is frozen during those held cycles, which is what suppresses the period jitter.

C. Lock Cycle Verification

An obvious concern is whether gear shifting slows acquisition. Table I lists the measured lock-cycle counts. For M = 1 and M = 2, the lock time equals the baseline for every K (28 and 33 cycles). For M = 4, it is 28 cycles for the baseline and for K = 4 and K = 8, and 31 cycles (3 more) for K = 1, 2 and 16. At the recommended operating point, acquisition latency is therefore unchanged.

TABLE I
Lock Acquisition Cycle Comparison

M	Baseline lock cycles	GS-ADPLL lock cycles	Acquisition impact
1	28	28 (all K)	None; identical
2	33	33 (all K)	None; identical
4	28	28 for K = 4, 8; 31 for K = 1, 2, 16	Matches baseline at K = 8

VI. ASIC Synthesis and Timing Analysis

Both RTL designs were synthesized in Cadence Genus with a commercial standard-cell library, using the same constraints (`constraints.sdc`) and optimization settings. Table II gives the quality-of-results (QoR) comparison.

A. Cell Count and Area Overhead

The baseline synthesizes to 51 leaf cells (37 combinational and 14 sequential) with a total area of 537.399 µm². The GS-ADPLL needs 81 leaf cells (58 combinational and 23 sequential) and 879.518 µm². That is +58.82% in cell count and +63.66% (+342.119 µm²) in area.

Most of the overhead comes from the state registers in `lock_detector_fsm.v` (`stable_count[7:0]`, `duty_count[3:0]` and the state bits). Sequential area rises by 68.55% (375.422 to 632.768 µm²) and combinational area by 54.21% (143.811 to 221.772 µm²). Fig. 3 shows that flip-flops are the largest part of the area.

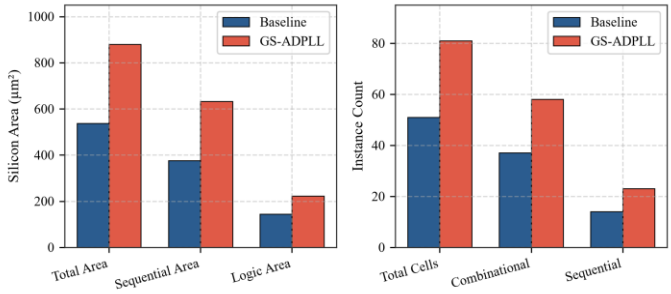


Fig. 3. Cell area (total, sequential and logic) and instance counts of the baseline ADPLL and the GS-ADPLL after synthesis in Cadence Genus.

B. Timing Slack and Critical Path

Static timing analysis in Genus shows that both designs meet setup and hold with large positive slack. Worst slack is 8472 ps for the baseline and 8075 ps for the GS-ADPLL, a small drop of 397 ps (−4.69%).

The worst data-path delay actually falls by 31.86%, from 1064 ps to 725 ps. This is attributed to the rewrite of `FILTER.v`: legacy asynchronous latch inferences were removed and unclocked multiplexing was replaced by fully synchronous enable gating, which gave Genus more room to restructure the logic (Fig. 4).

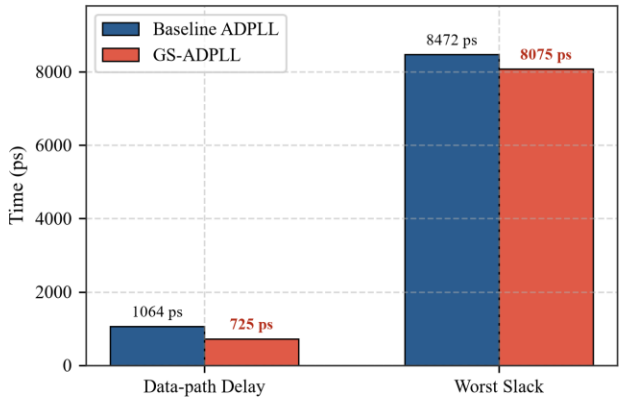


Fig. 4. Worst slack and critical data-path delay from Cadence Genus.

TABLE II
CADENCE GENUS SYNTHESIS QOR COMPARISON

Metric	Baseline	GS-ADPLL	Change
Total leaf instances	51	81	+58.82%
Sequential instances	14	23	+64.29%
Combinational instances	37	58	+56.76%
Total cell area	537.399 µm²	879.518 µm²	+63.66%
Sequential cell area	375.422 µm²	632.768 µm²	+68.55%
Logic cell area	143.811 µm²	221.772 µm²	+54.21%
Worst slack (met)	8472 ps	8075 ps	−4.69%
Worst data-path delay	1064 ps	725 ps	−31.86%
CPU runtime / peak memory	14 s / 1612 MB	14 s / 1613 MB	+0.03%

VII. POWER ANALYSIS

Power was evaluated at two levels: a static estimate from synthesis in Cadence Genus, and a vector-based dynamic analysis in Cadence Voltus-FI. The two flows answer different questions, so they are kept separate here.

A. Synthesis-Level Static Estimate (Genus)

Genus computes power from the Liberty characterization and a default statistical toggle rate (typically 0.1 toggles per cycle) applied to every net. With this model, total power rises by 64.50%, from 15.3919 µW to 25.3201 µW. Leakage goes from 2.604 to 4.316 µW (+65.72%), internal power from 11.207 to 18.527 µW (+65.31%), and switching power from 5.805 to 8.775 µW (+56.75%), as shown in Fig. 5.

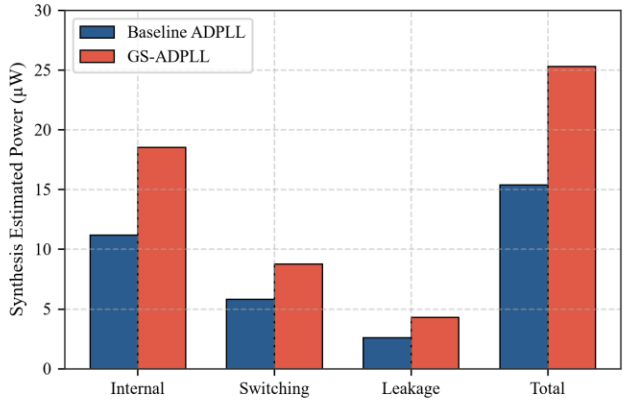


Fig. 5. Synthesis-level estimated power (leakage, internal, switching and total) from Cadence Genus.

This increase follows the +63.66% growth in area and the +64.29% growth in sequential instances. The estimate assumes that all 23 flip-flops toggle at the default rate, so it penalizes the added FSM logic and ignores the activity freezing that `compare_enable` performs.

B. Implementation-Level Vector-Based Analysis (Voltus)

Voltus combines the mapped gate-level netlist, back-annotated parasitics and cycle-accurate switching activity from Value Change Dump (VCD) vectors, so the real behavior of the gated loop filter is captured.

Table III and Fig. 6 show a very different picture. Total power goes from 0.585274 mW to 0.593940 mW, an increase of only 1.48% (+8.666 µW). Internal power rises by 1.74% (0.387644 to 0.394391 mW) and switching power by 0.30% (0.195025 to 0.195615 mW). Leakage power increases by 51.11% (0.002604 to 0.003935 mW), but it is a very small share of the total.

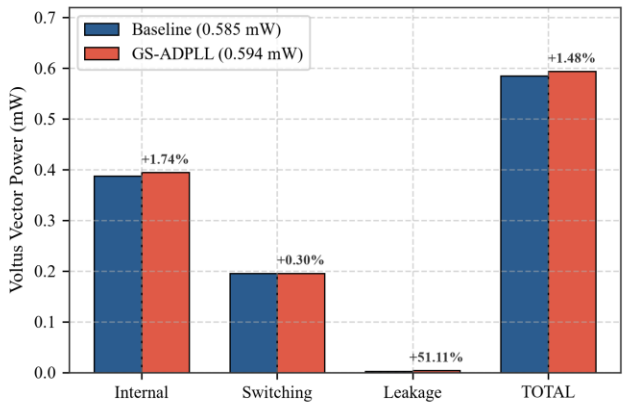


Fig. 6. Vector-based power (internal, switching, leakage and total) from Cadence Voltus.

TABLE III
CADENCE VOLTUS VECTOR-BASED POWER COMPARISON

Component	Baseline (mW)	GS-ADPLL (mW)	Delta (mW)	Change
Internal power	0.387644	0.394391	+0.006747	+1.74%
Switching power	0.195025	0.195615	+0.000590	+0.30%
Leakage power	0.002604	0.003935	+0.001331	+51.11%
Total power	**0.585274**	**0.593940**	**+0.008666**	**+1.48%**

C. Reconciling the Two Results

Fig. 7 sets the synthesis and Voltus results side by side. The GS-ADPLL does add gates, but at K = 8 the 87.5% activity reduction freezes the 4-bit up/down counters and the 8-bit average-code register for 7 of every 8 reference cycles. The switching saved in those nodes offsets the capacitance and power of the added FSM.

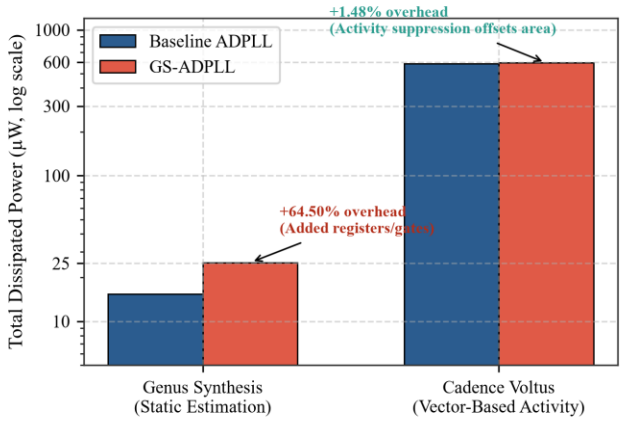


Fig. 7. Total power increase from static Genus estimation (+64.50%) and from Voltus vector-based analysis (+1.48%).

It should be stated plainly that the GS-ADPLL does not lower total power. The DCO and the divider run continuously to keep the clock oscillating. What the design provides is a jitter reduction of about 91% at essentially unchanged power (+1.48%).

VIII. PHYSICAL DESIGN AND IMPLEMENTATION

The synthesized netlist was implemented in Cadence Innovus. The flow covered floorplanning, power-grid creation (VDD/VSS rings and stripes), standard-cell placement, clock tree synthesis (CTS), NanoRoute detailed routing, and physical verification (DRC and LVS).

A. Floorplanning and Placement

The core was given a square aspect ratio with uniform placement rows, and power rings run along the core boundary to limit IR drop during peak switching. The 81 standard cells are spread evenly over the rows. The DCO delay-line stages were placed close together to limit interconnect parasitic mismatch between neighboring thermometer delay elements.

B. Routing and Layout

Detailed routing used the lower and intermediate metal layers. The final layout has no DRC violations, clean pin access and no local congestion hot spots. Fig. 8 shows the Innovus layout.

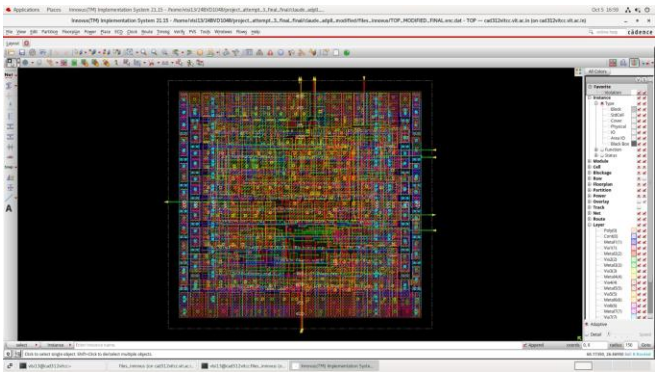


Fig. 8. Routed layout of the GS-ADPLL core in Cadence Innovus, showing placement rows, power grid and detailed routing.

C. Limitations

Core utilization, pin density and per-layer wirelength were not exported from the design environment, so they are not reported here.

IX. EXPERIMENTAL RESULTS AND PERFORMANCE EVALUATION

We ran a parametric simulation campaign to compare the jitter, lock behavior and frequency accuracy of the GS-ADPLL with the baseline over several operating conditions.

A. Baseline Jitter Performance

Table IV gives the baseline C2C jitter: 38.4 ps for M = 1, 18.8 ps for M = 2 and 7.8 ps for M = 4. The baseline frequency-tracking errors are 2.02%, 2.89% and 4.88%. Jitter falls as M grows because a higher division ratio averages the DCO period variation over more oscillator cycles. These three values are the reference for all comparisons below.

B. K-Factor Sweep

K was swept over {1, 2, 4, 8, 16} for M = 1, 2 and 4. Table IV lists the full results, and Figs. 9–12 show the trends.

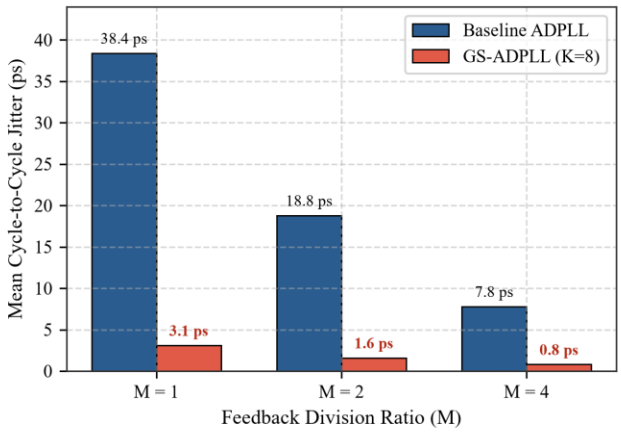


Fig. 9. C2C jitter of the baseline ADPLL and the GS-ADPLL (K = 8) for M = 1, 2 and 4.

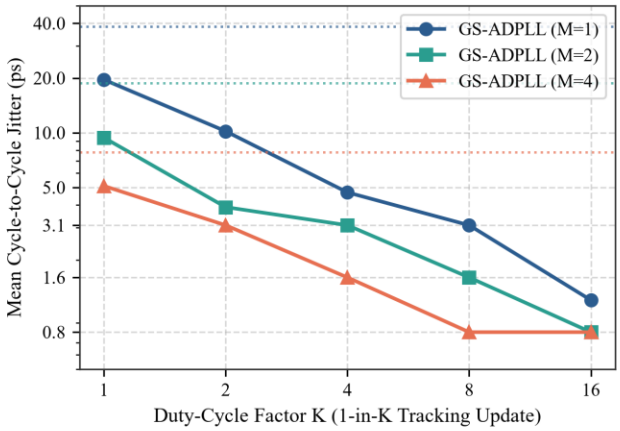


Fig. 10. Mean C2C jitter versus duty factor K (log scale) for M = 1, 2 and 4.

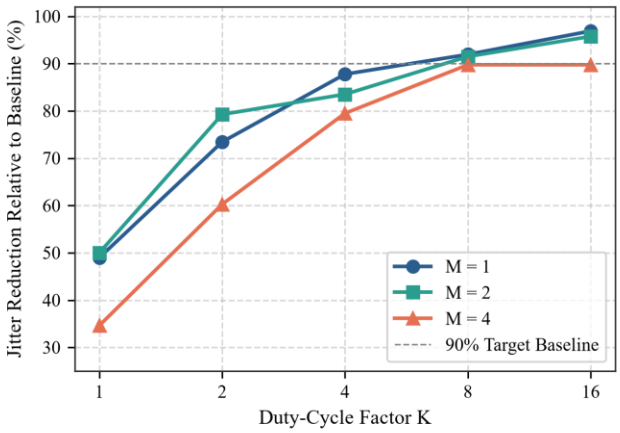


Fig. 11. C2C jitter reduction relative to the baseline over the K sweep.

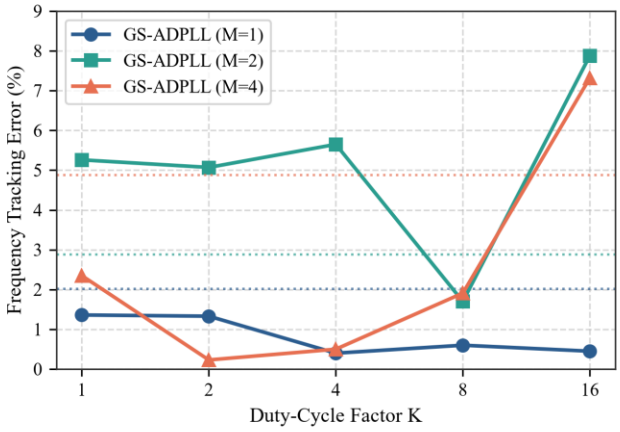


Fig. 12. Frequency tracking error (%) over the K sweep. The error grows at K = 16.

Several points stand out. First, at K = 1, where tracking activity is not reduced at all, C2C jitter already drops from 38.4 ps to 19.6 ps for M = 1 (48.96%), from 18.8 ps to 9.4 ps for M = 2 (50.00%), and from 7.8 ps to 5.1 ps for M = 4 (34.62%). Bandwidth gear shifting alone (Mechanism 1) therefore accounts for a large part of the gain, independent of duty cycling.

Second, as K increases from 1 to 8, Mechanism 2 reduces how often the DCO is perturbed. At K = 8, jitter is 3.1 ps for M = 1 (91.93%, 12.39× better), 1.6 ps for M = 2 (91.49%, 11.75×) and 0.8 ps for M = 4 (89.74%, 9.75×). The mean reduction over the three ratios is 91.05%, with 87.5% less tracking activity.

TABLE IV
PARAMETRIC K-FACTOR SWEEP RESULTS

M	K	Base jitter (ps)	GS jitter (ps)	Jitter red. (%)	Freq. error (%)	Activity red. (%)
1	1	38.4	19.6	48.96	1.36	0.0
1	2	38.4	10.2	73.44	1.33	50.0
1	4	38.4	4.7	87.76	0.40	75.0
1	**8**	**38.4**	**3.1**	**91.93**	**0.60**	**87.5**
1	16	38.4	1.2	96.88	0.45	93.8
2	1	18.8	9.4	50.00	5.26	0.0
2	2	18.8	3.9	79.26	5.07	50.0
2	4	18.8	3.1	83.51	5.65	75.0
2	**8**	**18.8**	**1.6**	**91.49**	**1.70**	**87.5**
2	16	18.8	0.8	95.74	7.88	93.8
4	1	7.8	5.1	34.62	2.35	0.0
4	2	7.8	3.1	60.26	0.23	50.0
4	4	7.8	1.6	79.49	0.50	75.0
4	**8**	**7.8**	**0.8**	**89.74**	**1.91**	**87.5**
4	16	7.8	0.8	89.74	7.32	93.8

C. Gear 2 Filter Threshold Sweep

To isolate the effect of the Gear 2 filter bandwidth, `CNT_N_GEAR2` was swept over {4, 6, 8, 10, 14} with K fixed at 8. Table V lists the results, and Figs. 13–15 show the trends.

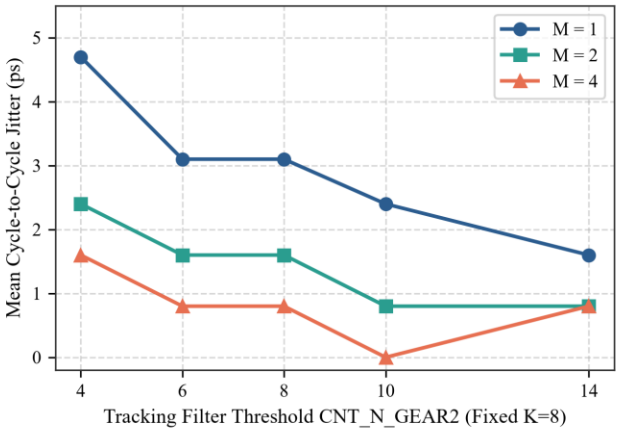


Fig. 13. C2C jitter versus Gear 2 threshold `CNT_N_GEAR2` (K = 8).

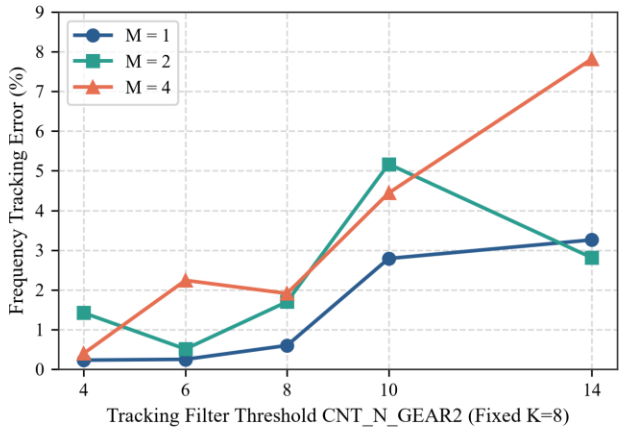


Fig. 14. Frequency tracking error (%) versus `CNT_N_GEAR2` (K = 8).

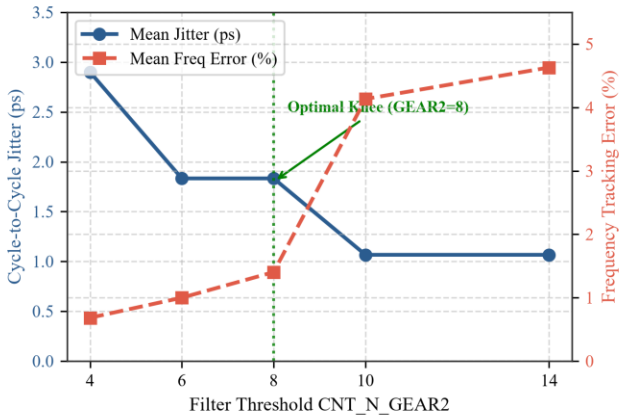


Fig. 15. Mean C2C jitter and frequency error versus `CNT_N_GEAR2`. GEAR2 = 8 is the best trade-off.

A higher threshold narrows the closed-loop bandwidth and lowers jitter to sub-picosecond levels (0.8 ps for M = 2 and M = 4 at GEAR2 = 14). The cost is tracking accuracy: for M = 4, the frequency error grows from 0.40% at GEAR2 = 4 to 7.82% at GEAR2 = 14. A threshold of 8 gives the best balance.

TABLE V
GEAR 2 FILTER THRESHOLD SWEEP RESULTS

M	GEAR2	Freq. error (%)	C2C jitter (ps)	Period drift (ps)	Activity red. (%)
1	4	0.23	4.7	−90.6	87.5
1	6	0.25	3.1	211.7	87.5
1	**8**	**0.60**	**3.1**	**−9.4**	**87.5**
1	10	2.79	2.4	−255.5	87.5
1	14	3.26	1.6	−96.1	87.5
2	4	1.42	2.4	−143.0	87.5
2	6	0.51	1.6	103.9	87.5
2	**8**	**1.70**	**1.6**	**−175.0**	**87.5**
2	10	5.17	0.8	−182.0	87.5
2	14	2.81	0.8	101.6	87.5
4	4	0.40	1.6	81.2	87.5
4	6	2.24	0.8	−127.3	87.5
4	**8**	**1.91**	**0.8**	**−101.6**	**87.5**
4	10	4.44	0.0	0.0	87.5
4	14	7.82	0.8	85.2	87.5

D. Choice of Operating Point (K = 8 versus K = 16)

K = 16 gives the lowest jitter in some cases (1.2 ps for M = 1 and 0.8 ps for M = 2 and M = 4), but the frequency error becomes too large: 7.88% for M = 2 and 7.32% for M = 4, with a 3-cycle lock penalty for M = 4. K = 8 keeps about 91% jitter reduction with bounded frequency error (0.60% for M = 1, 1.70% for M = 2, 1.91% for M = 4) and no lock-time penalty. We therefore select K = 8 as the operating point, as the trade-off plot in Fig. 16 shows.

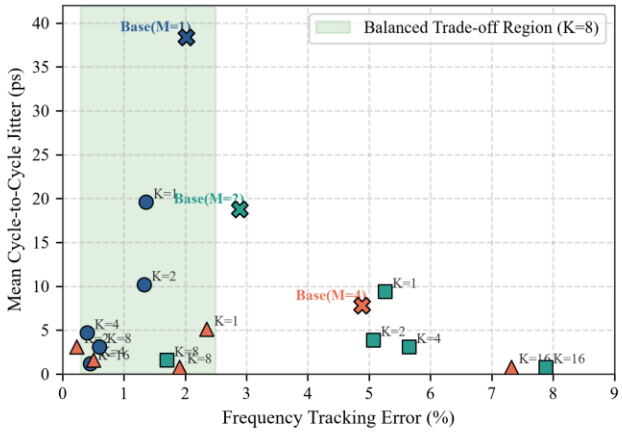


Fig. 16. Jitter versus frequency tracking error over the K sweep. K = 8 gives the best balance.

X. DISCUSSION

Fig. 17 summarizes the main benefits and costs of the design.

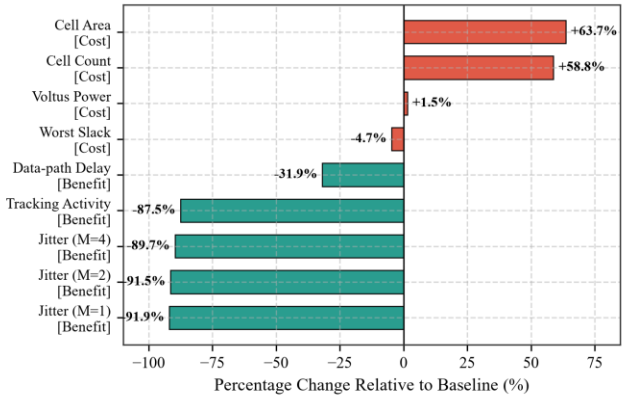


Fig. 17. Summary of benefits (jitter reduction, activity suppression, data-path speedup) against costs (area, sequential instances, Voltus power).

A. Engineering Trade-Offs

The GS-ADPLL cuts C2C jitter by about 91% and loop-filter activity by 87.5% without lengthening lock time. The main cost is silicon area. The FSM, duty counter and stability logic add 342 µm² (+63.66%) and 30 cells (+58.82%). In an SoC with dense digital logic, 342 µm² is a small price for a roughly tenfold improvement in clock timing precision.

B. Power Overhead

A common assumption is that extra digital control logic must raise chip power noticeably. The synthesis-level estimate

agreed (+64.50%), but the Voltus vector-based result shows only +1.48%. Duty-cycled gating of the loop-filter registers removes enough switching activity to cancel most of the power of the new control registers. Activity-aware clock or data gating can therefore make such control additions nearly power-neutral at the SoC level.

C. Limitations and Future Work

Some limits should be noted. The experiments measured C2C period jitter and long-term drift only. Spectral phase noise L(f) and integrated RMS phase jitter still need transistor-level periodic steady-state (PSS) and periodic noise (PNOISE) simulation. Timing and power results are at nominal PVT, so multi-corner signoff over process corners (SS, FF) and temperatures from −40 °C to 125 °C is needed before tape-out. Finally, extending the FSM to a multi-gear controller (for example Gears 1 to 3 with dynamic duty-factor changes) could improve the trade-off between jitter and tracking accuracy further.

XI. Conclusion

This paper presented the design, synthesis, layout, power evaluation and jitter characterization of a gear-shifting all-digital PLL. Once lock is confirmed, a lock-state FSM raises the loop-filter threshold from 4 to 8 and limits phase comparison to one in K reference cycles, so the loop can lock quickly and then run quietly. At K = 8, steady-state C2C jitter drops by 91.93% (M = 1), 91.49% (M = 2) and 89.74% (M = 4) against the baseline, with 87.5% less tracking activity and no lock-time penalty. The design costs 63.66% more area, adds only 1.48% to total power in vector-based analysis, and shortens the critical data path by 31.86%. These results make the GS-ADPLL a practical, synthesizable option for low-jitter clock generation in low-power IoT and SoC designs.

References

[1] B. Razavi, *Design of Integrated Circuits for Optical Communications*, 2nd ed. Hoboken, NJ, USA: Wiley, 2012.

[2] R. E. Best, *Phase-Locked Loops: Design, Simulation, and Applications*, 6th ed. New York, NY, USA: McGraw-Hill, 2007.

[3] J.-C. Liu and R.-C. Ai, “A low-voltage and low-power PLL for sub-GHz IoT applications,” *Integration, VLSI J.*, vol. 103, Art. no. 102424, 2025, doi: 10.1016/j.vlsi.2025.102424.

[4] R. B. Staszewski and P. T. Balsara, *All-Digital Frequency Synthesizer in Deep-Submicron CMOS*. Hoboken, NJ, USA: Wiley, 2006.

[5] C.-C. Chung and C.-Y. Lee, “An all-digital phase-locked loop for high-speed clock generation,” *IEEE J. Solid-State Circuits*, vol. 38, no. 2, pp. 347–351, Feb. 2003, doi: 10.1109/JSSC.2002.807400.

[6] J. A. Tierno, A. V. Rylyakov, and D. J. Friedman, “A wide power supply range, wide tuning range, all static CMOS all digital PLL in 65 nm SOI,” *IEEE J. Solid-State Circuits*, vol. 43, no. 1, pp. 42–51, Jan. 2008, doi: 10.1109/JSSC.2007.910972.

[7] T.-Y. Hsu, C.-C. Wang, and C.-Y. Wu, “A low-jitter ADPLL for clock generation,” *IEEE J. Solid-State Circuits*, vol. 36, no. 8, pp. 1264–1273, Aug. 2001, doi: 10.1109/4.933795.

[8] W. Deng *et al.*, “A 0.0066 mm² 75% duty-cycle digitally controlled oscillator for low-power fast-locking ADPLL,” *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 60, no. 10, pp. 637–641, Oct. 2013, doi: 10.1109/TCSII.2013.2273836.

[9] Motorola, Inc., “Digital loop filter and phase locked loop using the same,” U.S. Patent 5 506 875, Apr. 9, 1996.